CHAPTER 17: Hierarchical clustering

🌳 [17.1] Hierarchical Agglomerative Clustering (HAC)

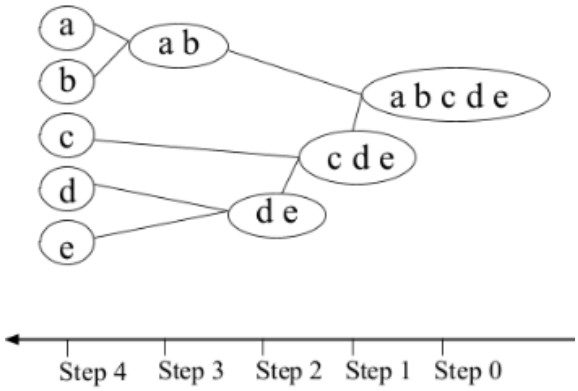
- Hierarchical clustering builds a **tree of clusters** instead of making just one flat grouping like K-means.
- This structure is more informative
- doesnt require us to pre-specify number of clusters

🌲 Main Idea of Hierarchical Clustering

There are **two approaches**:

🔼 Top-Down Clustering [divisive approach]

- Start with: **one giant cluster** containing all documents
- Then: **repeatedly split clusters** into smaller ones
- Until: each document becomes separate

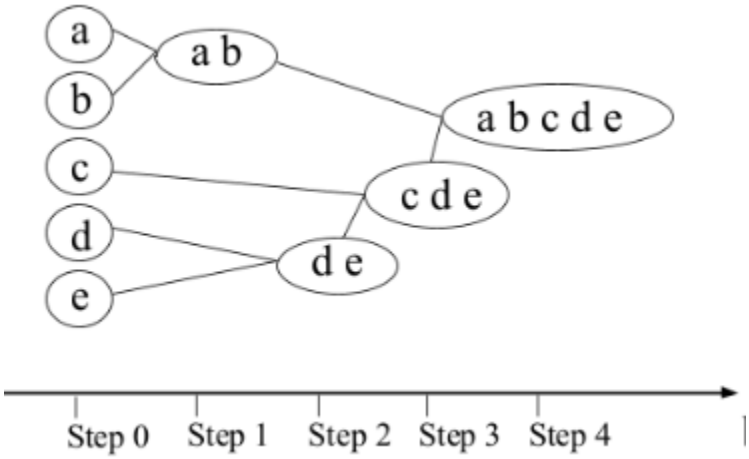


🔽 Bottom-Up Clustering (HAC)

- Start with: every document as its own cluster (**singleton cluster**)
- Then: **repeatedly merge** the most similar clusters
- Until: all documents become **one giant cluster**

This is called **Hierarchical Agglomerative Clustering (HAC)**

"Agglomerative" = merging together.



Agglomerative clustering

- ↳ bottoms up method
- ↳ starts with each eg in its own cluster
- ↳ iteratively combines them
- ↳ to form larger clusters

Divisive clustering

- ↳ Partitional / top down method
- ↳ seperates all eg immediately into clusters

🌳 Dendrogram

A dendrogram is the **tree diagram used to show HAC.**

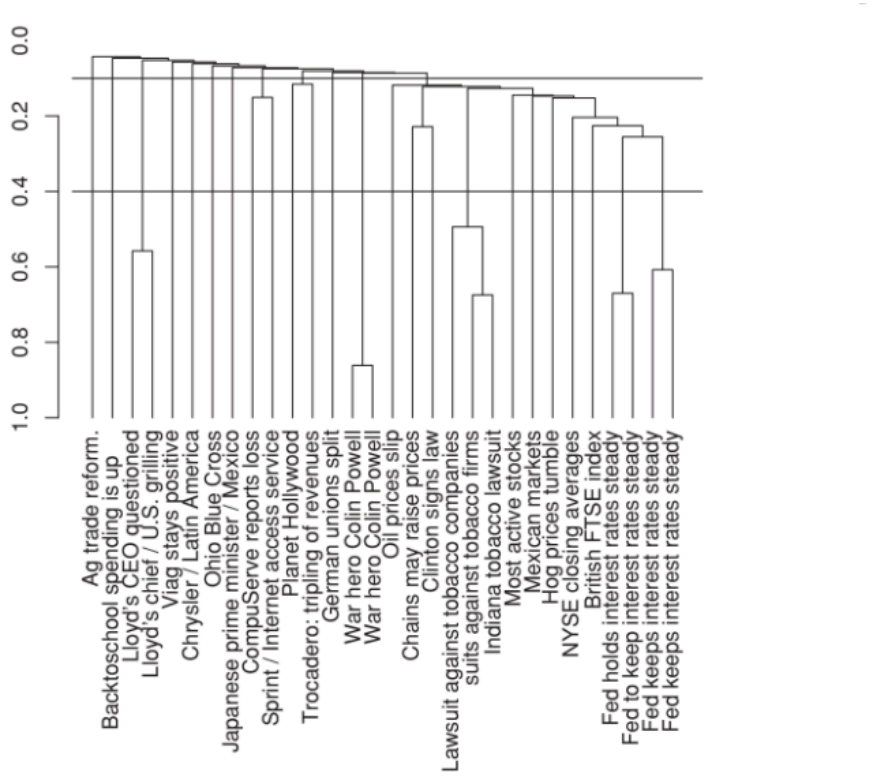


Figure 17.1 A dendrogram of a single-link clustering of thirty documents from Reuters-RCV1. Two possible cuts of the dendrogram are shown: at 0.4 into twenty-four clusters and at 0.1 into twelve clusters.

- Each horizontal line = a **merge between clusters**.
- The height of the line = **similarity at which merge happened**.

- Each node = a **cluster**
- Each leaf node = **singleton cluster**

🔹 Lower merges

Documents **merged very early** (near bottom) → are **VERY similar.**

🔹 Higher merges

Documents **merged later** (near top) → are **less similar**.

The two "War hero Colin Powell" documents merge first.

Why? Because they are **extremely similar.**

So their merge happens: **low in the tree at high similarity**

[the similarity at which they merge at is called ⇒ **combination similarity**]

✂️ Cutting the Dendrogram

HAC gives a **hierarchy/tree.**

But sometimes we need normal clusters, So we "cut" the tree horizontally.

✂️ Cut at High Similarity

If we cut high up:

- **many small clusters form**
- **only very similar docs stay together**

Example:

Cut at 0.4 → 24 clusters

✂️ Cut at Low Similarity

If we cut lower:

- **bigger clusters form**
- less similar docs can join

Example:

Cut at 0.1 → 12 clusters

🌳 Higher cut

= stricter clustering
= more clusters

🌳 Lower cut

= looser clustering
= fewer clusters

✅ Monotonic HAC

Suppose merge similarities are: 0.9 → 0.7 → 0.5 → 0.3

Each merge becomes LESS similar over time.

This makes sense because:

- **easiest merges happen first**
- harder merges happen later

This is called: **Monotonicity**

❌ Inversion

Suppose: 0.5 → 0.8

This means:

- **later merge is MORE similar than earlier merge**

That is weird.

This is called: **Inversion**

Usually undesirable.

🧠 HAC Algorithm

```
SIMPLEHAC(d_1, ..., d_N)
 1  for n ← 1 to N
 2  do for i ← 1 to N
 3     do C[n][i] ← SIM(d_n, d_i)
 4     I[n] ← 1  (keeps track of active clusters)
 5  A ← []  (collects clustering as a sequence of merges)
 6  for k ← 1 to N − 1
 7  do ⟨i, m⟩ ← arg max_{⟨i,m⟩: i≠m ∧ I[i]=1 ∧ I[m]=1} C[i][m]
 8     A.APPEND(⟨i, m⟩)  (store merge)
 9     for j ← 1 to N
10     do C[i][j] ← SIM(i, m, j)
11        C[j][i] ← SIM(i, m, j)
12     I[m] ← 0  (deactivate cluster)
13  return A
```

Line 1:

Loop through all documents/data points.

Line 2:

For each document, compare it with every other document.

Line 3:

Compute similarity between documents `dn` and `di` and store it in similarity matrix `C`.

Line 4:

Set `I[n] = 1` to mark every cluster as initially active.

(Initially, each document is its own cluster.)

Line 5:

Create list `A` to store the sequence/history of cluster merges.

Line 6:

Repeat merging process `N - 1` times until only one cluster remains.

Line 7:

Find the two active clusters `(i, m)` having the highest similarity:

$$\langle i, m \rangle = \arg\max_{\{(i,m):i \neq m \wedge I[i]=1 \wedge I[m]=1\}} C[i][m]$$

Line 8:

Store this merge `(i, m)` in list `A`.

Line 9:

Loop through all clusters again to update similarities after merging.

Line 10:

Update similarity between new merged cluster and other clusters.

Line 11:

Since similarity matrix is symmetric, update the reverse entry too.

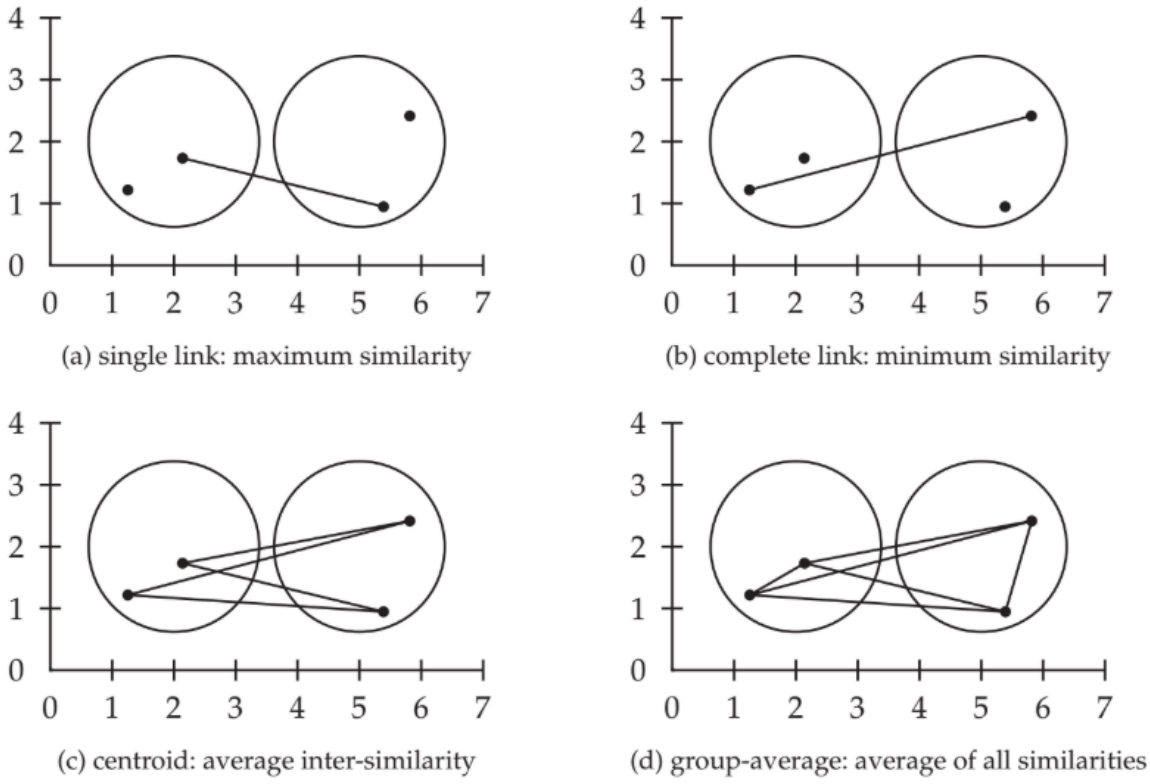
Line 12:

Mark cluster `m` as inactive because it has been merged into cluster `i`.

Line 13:

Return `A`, which contains the full hierarchical clustering sequence (dendrogram information).

🔥 Four Types of HAC



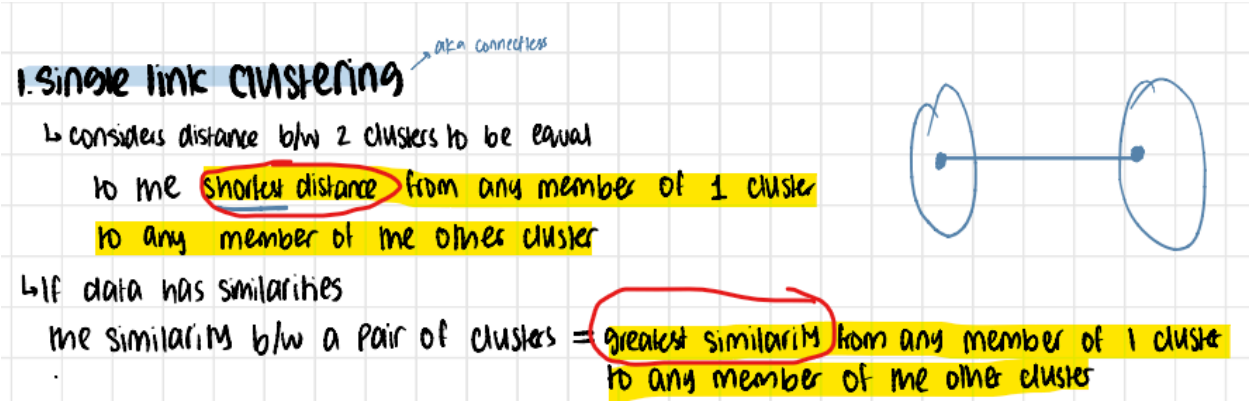
(a) single link: maximum similarity

(b) complete link: minimum similarity

(c) centroid: average inter-similarity

(d) group-average: average of all similarities

1️⃣ Single-Link Clustering



Uses: **maximum similarity**

Looks for: 👉 **closest pair only.**

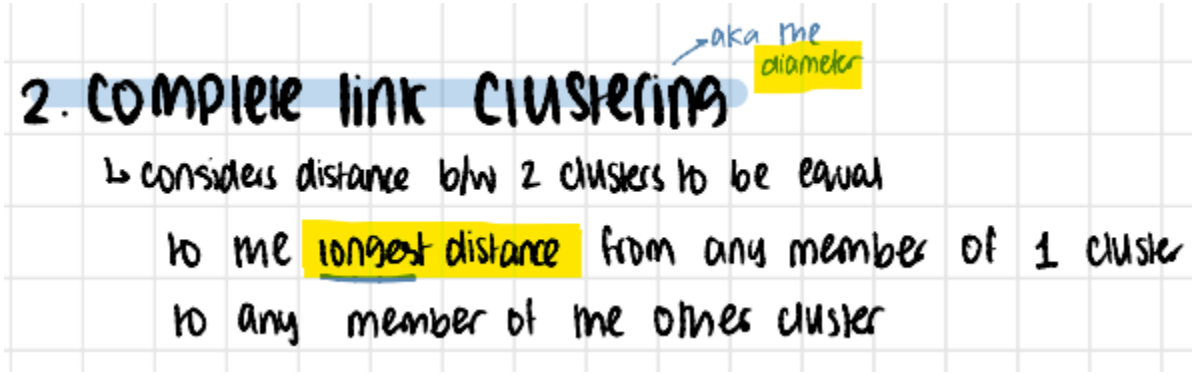
sim(A,B) = max(sim(x,y))

where:

- x in A
- y in B

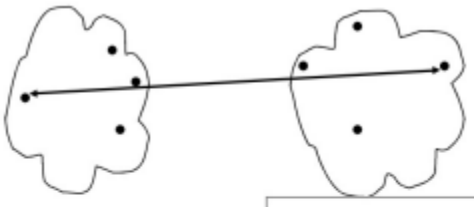
---

2 Complete-Link Clustering



Uses: **minimum similarity**

sim(A,B) = min(sim(x,y))

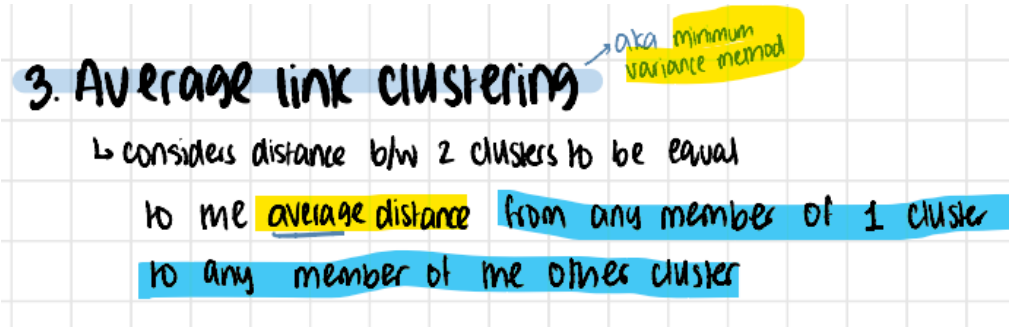


- Single-link
- Complete-link
- Average-link
- Centroid distance

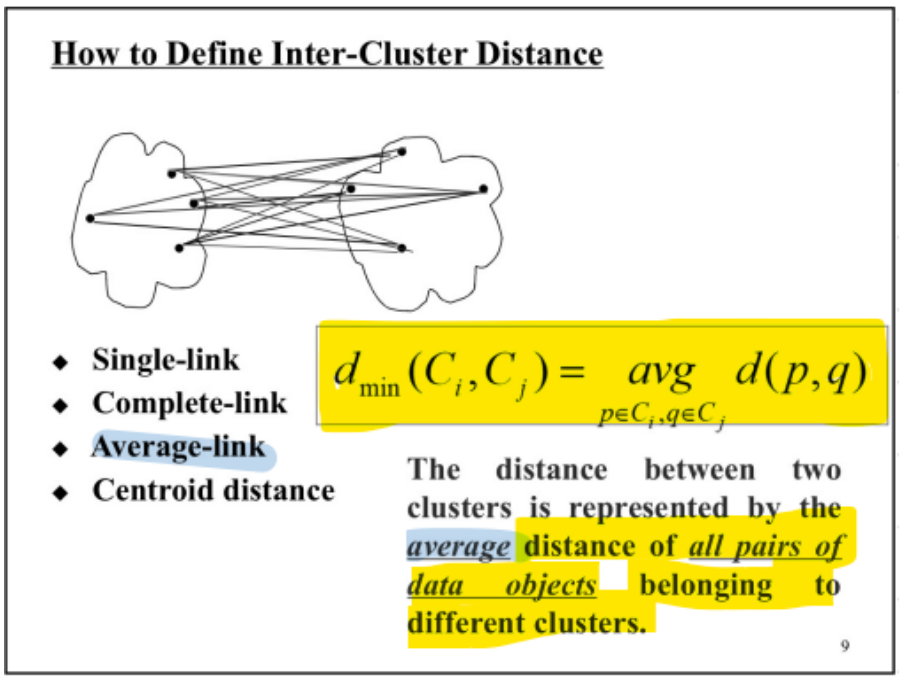
$$d_{\min}(C_i, C_j) = \min_{p \in C_i, q \in C_j} d(p, q)$$

The distance between two clusters is represented by the distance of the *farthest pair of data objects* belonging to different clusters.

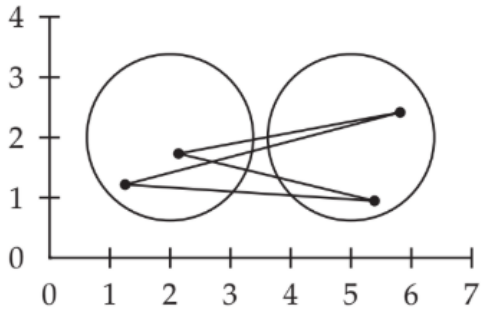
3 Group-Average Clustering



Uses: **average similarity between all pairs.**

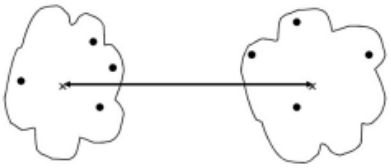


4 Centroid Clustering



(c) centroid: average inter-similarity

How to Define Inter-Cluster Distance



m_i, m_j are the means of C_i, C_j,

- **Single-link**
- **Complete-link**
- **Average-link**
- **Centroid distance**

$$d_{mean}(C_i, C_j) = d(m_i, m_j)$$

The distance between two clusters is represented by the distance between *the means of the cluters.*

10

⚡ Difference Between HAC and K-Means

K-Means	HAC
Flat clustering	**Hierarchical** clustering
Need K beforehand	No need initially
Uses **centroids**	Uses **pairwise merging**
Reassigns docs **repeatedly**	**Once merged**, **never split**
Fast	**Slower**

Output = **clusters** Output = **tree**

🚨 Important HAC Property

› Once merged → always merged

- HAC **NEVER undoes merges.**
- This is a major difference from K-means.
- K-means keeps moving documents around.
- HAC does not.

🔗 [17.2] Single-Link vs Complete-Link Clustering

The BIG difference between them is: 👉 **How they measure similarity between clusters.**

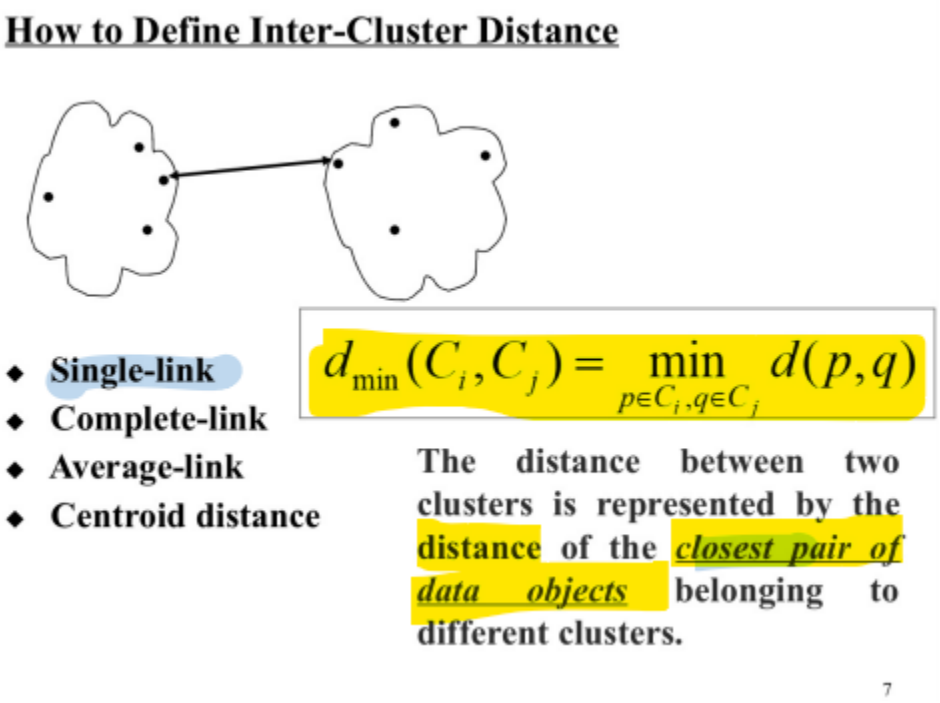
🔗 Single-Link Clustering

- In single-link clustering, two clusters are considered close if **even one pair of documents between them** is **very similar.**
- So we only care about the **MOST similar pair** between the two clusters.
- the **similarity of two clusters** is the **similarity of their most similar members**

📌 Formula Idea

If clusters are A and B:

sim(A,B) = maximum similarity between any pair



- Min distance = max similarity

🖼️ Figure 17.4

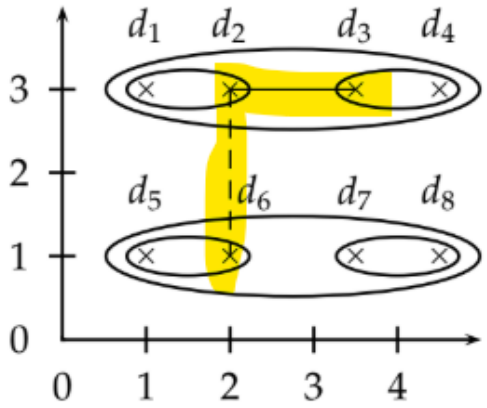


Figure 17.4 A single-link (*left*) and complete-link (*right*) clustering of eight documents. The ellipses correspond to successive clustering stages. *Left*: The single-link similarity of the two upper two-point clusters is the similarity of d_2 and d_3 (*solid line*), which is greater than the single-link similarity of the two left two-point clusters (*dashed line*). *Right*: The complete-link similarity of the two upper two-point clusters is the similarity of d_1 and d_4 (*dashed line*), which is smaller than the complete-link similarity of *the two left* two-point clusters (*solid line*).

Initially:

(d1,d2)
(d3,d4)
(d5,d6)
(d7,d8)

These nearby pairs merge first.

Then single-link checks: 👉 **Which two clusters contain the closest pair?**

The closest pair is: **d2 and d3**

So the **two upper clusters merge**.

Even though: **d1 and d4 are farther apart.**

Single-link ignores that.

It only cares about the BEST connection.

⚠️ Why Single-Link Can Be Bad

Because: **one close connection is enough**

clusters can **grow into long chains.**

This is called: ⛓️ **Chaining Effect**

🖼️ Figure 17.6 (Chaining)

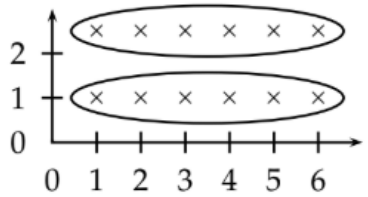


Figure 17.6 Chaining in single-link clustering. The local criterion in single-link clustering can cause undesirable elongated clusters.

Notice the long stretched clusters? **Each point is close to the next point.**

So the cluster **keeps extending**.

Eventually: **documents very far apart** end up in the **same cluster.**

✅ Strength of Single-Link

Good at:

- **detecting irregular shapes**
- **connected regions**

❌ Weakness

Creates:

- **loose**
- **stretched**
- **messy clusters**

🧱 Complete-Link Clustering

Complete-link clustering is the **opposite**.

Two clusters merge only if: Their **MOST DISSIMILAR pair is still reasonably similar.**

So now we care about: 👉 the WORST pair.

📌 Formula Idea

sim(A,B) = minimum similarity between pairs

(or equivalently: **smallest maximum distance**)nono

🖼️ Figure 17.4 (Right Side)

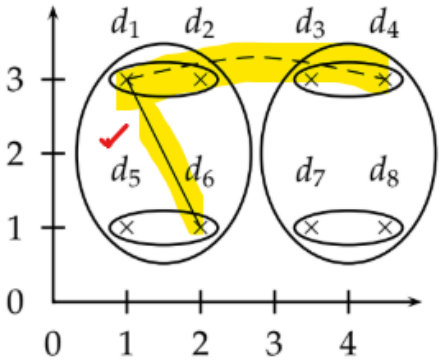


Figure 17.4 A single-link (*left*) and complete-link (*right*) clustering of eight documents. The ellipses correspond to successive clustering stages. *Left*: The single-link similarity of the two upper two-point clusters is the similarity of d_2 and d_3 (*solid line*), which is greater than the single-link similarity of the two left two-point clusters (*dashed line*). *Right*: The complete-link similarity of the two upper two-point clusters is the similarity of d_1 and d_4 (*dashed line*), which is smaller than the complete-link similarity of *the two left* two-point clusters (*solid line*).

(d1,d2)
(d3,d4)
(d5,d6)
(d7,d8)

Now complete-link asks: 👉 **Which merge keeps the cluster most compact?**

It merges: left two pairs, instead of upper pairs.

Why? Because if upper pairs merged:

- d1 and d4 are very far apart.

That would create a **wide cluster**.

Complete-link avoids that.

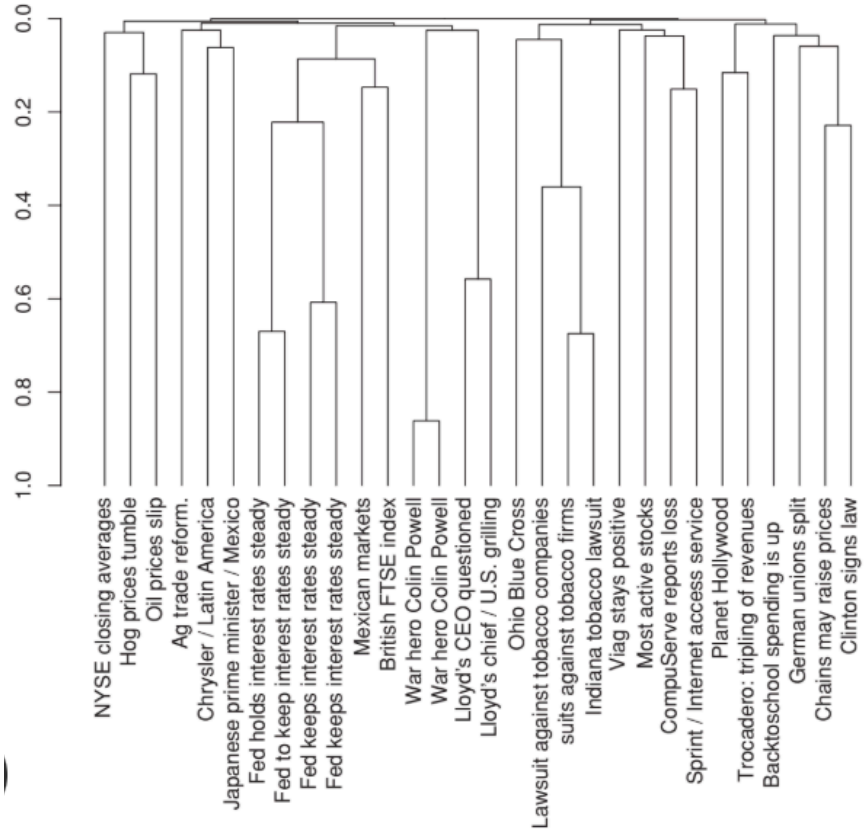
✅ Result

Complete-link prefers: **Compact, tight clusters**

- Prevents long chains

🖼️ Figure 17.5

This is the complete-link dendrogram.



Notice:

- **clusters are more balanced**
- **cleaner separation occurs**

Compared to Figure 17.1 (single-link),
which had chain-like merging.

⚠️ Problem with Complete-Link

Complete-link is **VERY sensitive to outliers**

🌟 **Outlier** 👉 a point far away from others.

🖼️ Figure 17.7

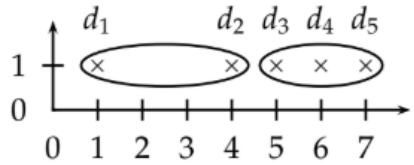


Figure 17.7 Outliers in complete-link clustering. The five documents have the x-coordinates $1 + 2\epsilon$, 4, $5 + 2\epsilon$, 6 and $7 - \epsilon$. Complete-link clustering creates the two clusters shown as ellipses. The most intuitive two-cluster clustering is $\{\{d_1\}, \{d_2, d_3, d_4, d_5\}\}$, but in complete-link clustering, the outlier d_1 splits $\{d_2, d_3, d_4, d_5\}$ as shown.

Documents **d2 d3 d4 d5** naturally belong together.

But: **d1** is **far away.**

Because complete-link **looks at the WORST distance**,
the **outlier ruins the merge.**

🧠 Why?

Suppose we merge:

{d1,d2}
with
{d3,d4,d5}

Complete-link checks: 👉 **farthest pair.**

That pair is: d1 and d5

Huge distance.

So **merge is rejected.**

Single-Link	Complete-Link
Uses **closest pair**	Uses **farthest pair**
Local criterion	**Global criterion**
Allows chains	**Prefers compact clusters**
Robust to outliers	**Sensitive** to **outliers**
Loose clusters	**Tight clusters**

📍 Local Criterion vs Global Criterion

This describes:👉 **How much of the cluster structure the algorithm looks at before merging clusters.**

🔗 Single-Link = Local Criterion

Single-link only looks at: **ONE local connection** between two clusters.

Specifically:the **closest pair of points**.

It ignores everything else.

the decision depends on a **small local area** of the clusters.

🧱 Complete-Link = Global Criterion

Complete-link looks at: **ENTIRE cluster structure** before merging.

It checks:

- the **farthest pair between the clusters.**

So **every document matters**.

📌 Why "Global"?

Because:

- the **whole cluster arrangement affects the decision.**

Not just one tiny connection.

⚡ Result of This Difference

Local (Single-Link)	Global (Complete-Link)
Easy merging	**Strict** merging
Long chains possible	**Compact clusters**
Ignores overall shape	**Considers overall shape**

⭐ [17.2.1] Time Complexity — Easy Explanation

The main issue in HAC (Hierarchical Agglomerative Clustering) is: **"At every step, how do we quickly find the two clusters that should merge next?"**

1. Why naive HAC is slow — $O(N^3)$

In HAC:

- Start with **(N) single-document clusters**.
- Repeatedly:
    1. Find the **most similar pair**
    2. **Merge** them
    3. **Update similarities**

At each iteration:

- You search the whole similarity matrix (C)
- Matrix size ≈ (**N x N**)

That costs:

$$O(N^2)$$

And you do this about: **N-1** times.

So total:

$$O(N^3)$$

---

2. Faster HAC using Priority Queues — $O(N^2 \log N)$

Instead of scanning the whole matrix every time:

For each cluster, keep its "**best merge candidate**" **already sorted.**

That structure is called a: **Priority Queue**

---

3. Similarity Matrix (C)

The matrix stores **pairwise similarities**.

	1	2	3	4
1	-	0.7	0.2	0.1
2	0.7	-	0.8	0.3
3	0.2	0.8	-	0.5
4	0.1	0.3	0.5	-

meaning::

C[2][3] = 0.8

means **cluster 2 and 3 are highly similar.**

---

4. What Priority Queue stores

For each row: Instead of rescanning: **Store clusters ordered by similarity.**

Example for cluster 2:

Candidate	Similarity
3	0.8
1	0.7
4	0.3

Then:

$$P[2].max()$$

instantly gives: Cluster 3

---

5. Why this is faster

Operations like:

- Insert, delete, Update, get max

Take: $O(\log N)$

instead of rescanning everything.

So overall complexity becomes: $O(N^2 \log N)$

---

6. Different HAC methods use different similarity formulas

- The algorithm framework is same.
- **Only the similarity formula changes.**

Single-link

$$\max(\text{sim}(i, k_1), \text{sim}(i, k_2))$$

Meaning: Take the **MOST similar pair.**

Complete-link

$$\min(\text{sim}(i, k_1), \text{sim}(i, k_2))$$

Meaning: Take the **WORST pair similarity.**

clustering algorithm	$\text{SIM}(i, k_1, k_2)$
single-link	$\max(\text{SIM}(i, k_1), \text{SIM}(i, k_2))$
complete-link	$\min(\text{SIM}(i, k_1), \text{SIM}(i, k_2))$
centroid	$(\frac{1}{N_m}\vec{v}_m) \cdot (\frac{1}{N_i}\vec{v}_i)$
group-average	$\frac{1}{(N_m+N_i)(N_m+N_i-1)}[(\vec{v}_m + \vec{v}_i)^2 - (N_m + N_i)]$

compute $C[5]$	1	2	3	4	5
	0.2	0.8	0.6	0.4	1.0
create $P[5]$ (by sorting)	2	3	4	1	
	0.8	0.6	0.4	0.2	
merge 2 and 3, update similarity of 2, delete 3	2	4	1		
	0.3	0.4	0.2		
delete and reinsert 2	4	2	1		
	0.4	0.3	0.2		

Centroid

Uses centroid vectors.

Similarity depends on cluster centers.

Group-average

Uses average similarity over all pairs.

Why Single-Link Is Easy to Optimize

- Suppose **cluster A's best merge candidate is cluster B.**

- Now **B merges with** another **cluster C**.
- In single-link, the **new merged cluster (B+C)** will usually **STILL be A's best candidate.**
- Why?
    - Because **single-link only cares about the strongest connection**.
- That strongest connection usually **survives the merge.**
- So we **do NOT need to recompute** everything again.
- This property is called: **Best-Merge Persistence**

Meaning: "**The best merge relationship tends to persist after merging.**"

NBM Array (Next Best Merge)

- Because of best-merge persistence, we can store: "**What is the best cluster for me to merge with?"**

  inside something called the **NBM array.**

- makes single-link very efficient: $O(N^2)$

😞 Why this does NOT work for Complete-Link

- Instead of looking at the BEST pair between clusters, it **looks at the WORST pair.**
- Two clusters are similar only if: **ALL points inside them are reasonably close.**
- So complete-link cares about the **ENTIRE cluster structure**.

- This is called a: **Global or Nonlocal Criterion** because faraway points matter too.
- So old "best merge" information becomes invalid.
- We must recompute similarities again.
- Thus no NBM optimization.
- So complete-link stays: $O(N^2 \log N)$

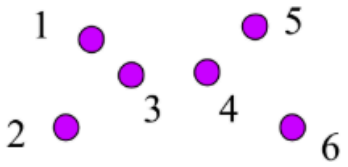
⭐Final Big Picture

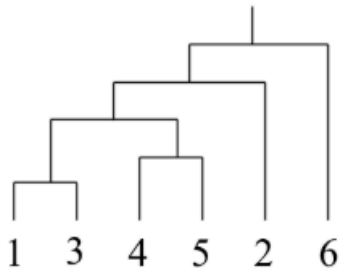
Algorithm	Complexity	Why
Naive HAC	$O(N^3)$	Full matrix scan every iteration
HAC + Priority Queue	$O(N^2 \log N)$	Faster similarity lookup
Single-Link + NBM	$O(N^2)$	Best-merge persistence

---

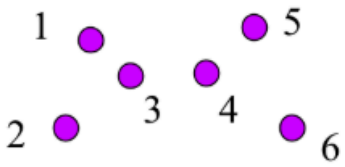
http://drive.google.com/file/d/1ntJbrQruxWSShhVZMETzSyQ_ztAD14-X/view

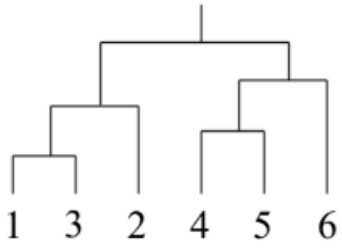
Result of the Single-Link algorithm





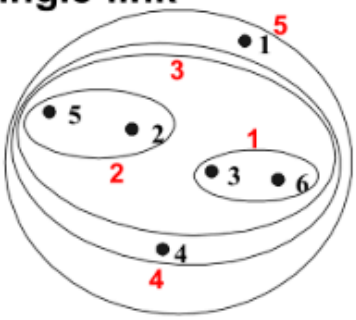
Result of the Complete-Link algorithm



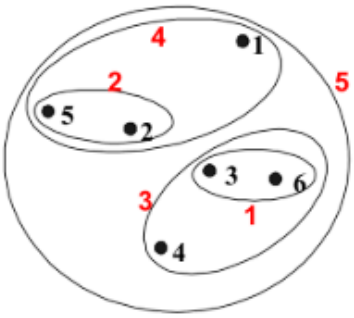


Hierarchical Clustering: Comparison

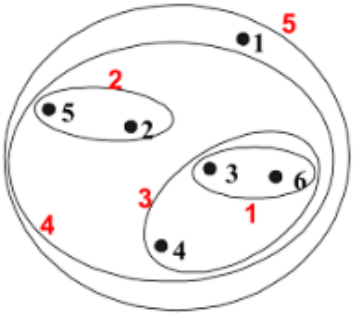
Single-link



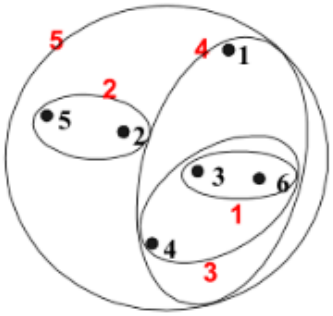
Complete-link



Average-link

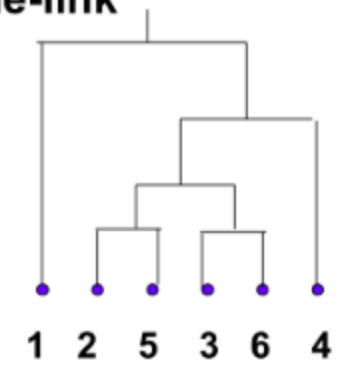


Centroid distance

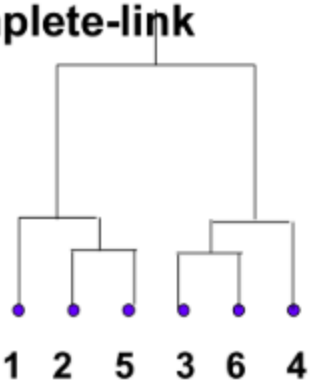


Compare Dendrograms

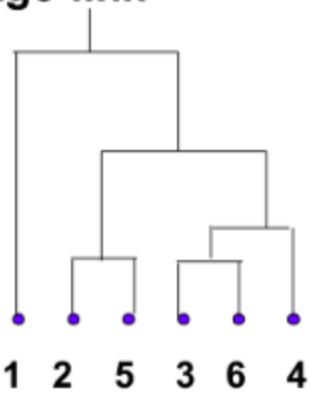
Single-link



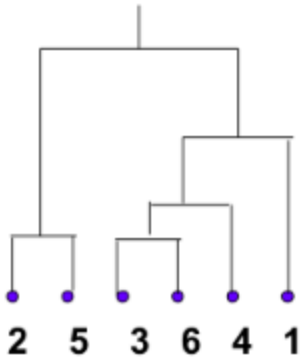
Complete-link



Average-link



Centroid distance



[^ kuch khaas smjh nhi aayi yeh bs daaldi cause its in slides]

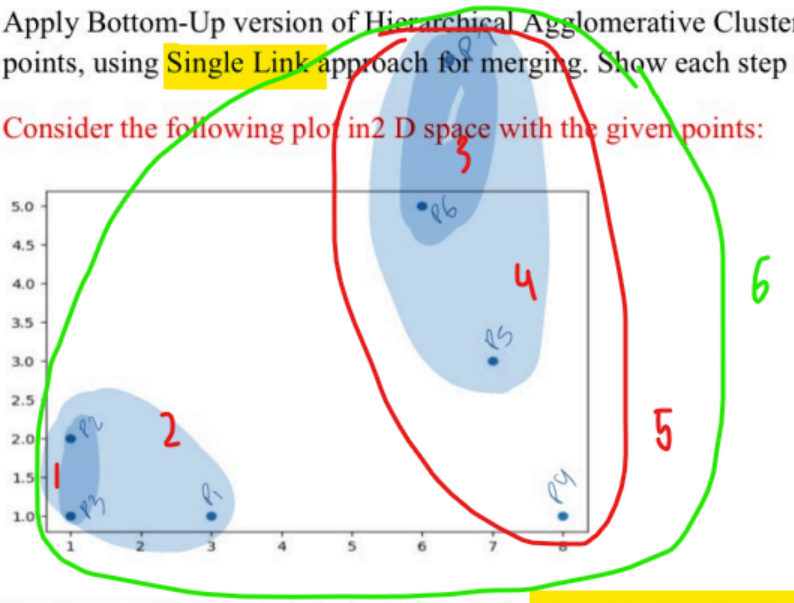
Question No. 2

Consider the following data points in 2D space.

P1(3,1), P2(1,2), P3(1,1), P4(8,1), P5(7,3), P6(6,5), P7(6,7)

Apply Bottom-Up version of Hierarchical Agglomerative Clustering (HAC) for the given data points, using Single Link approach for merging. Show each step of the algorithm

Consider the following plot in2 D space with the given points:



For Bottom Up Hierarchal clustering, Point P3 and P2 are more similar and will be merged first.

Second Point P1 will be merged with the cluster C1(P2, P3) to produced C2(P1, C1).

Thirdly, we will merge P6 and P7 to produced C3(P6, P7) It will merge later P5 to produced C4(C3, P5). Similarly, it will merge the P4 to it to produced C5(C4, P4) and in the final merge C2 and C5 will be merged.

a. Differentiate between k-Mean and Agglomerative Hierarchical clustering algorithms.

1. Hierarchical and Partitional(k-Mean) Clustering have key differences in running time, assumptions, input parameters and resultant clusters.
2. k-Mean clustering is faster than hierarchical clustering.
3. Hierarchical clustering requires only a similarity measure, while k-Mean clustering requires stronger assumptions such as number of clusters and the initial centers.
4. Hierarchical clustering does not require any input parameters, while k-Mean clustering algorithms require the number of clusters to start running.
5. Hierarchical clustering returns a much more meaningful and subjective division of clusters but k-Mean clustering results in exactly k clusters.
6. Hierarchical clustering algorithms are more suitable for categorical data as long as a similarity measure can be defined accordingly.

‹food for thoughts›

1. What is Hierarchical Clustering? what are its benefits?

- A clustering method that builds a **hierarchy/tree of clusters.**
- Represented using a **dendrogram**.
- Can be:
    - **Bottom-up (HAC)** → merge clusters
    - **Top-down** → split clusters

Benefits

- No need to predefine number of clusters K
- **Shows hierarchical relationships between documents**
- Easy to visualize using dendrograms
- Allows **clustering at different levels** by **cutting** the **dendrogram**
- Useful for exploratory analysis

2. Define How Hierarchical Agglomerative Clustering (HAC) algorithm works.

HAC Algorithm

1. Start with each document as its own cluster.
2. **Compute similarities between all clusters.**
3. **Merge** the two **most similar clusters.**
4. **Update similarities.**
5. Repeat until:
    - one cluster remains

Output

- Produces a dendrogram showing merge history.

3. How are the different variations of HAC produced? Explain each one briefly.

Single-Link Clustering

- Similarity = similarity of the **closest pair** of documents.
- Uses a **local criterion**.
- Can produce **chaining** (long stretched clusters).

Complete-Link Clustering

- Similarity = similarity of the **most distant pair** of documents.
- Uses a **global criterion**.
- Produces **compact** clusters.
- **Sensitive to outliers.**

Centroid Clustering

- Uses cluster centroids (mean vectors).
- **Similarity based on centroid distance.**
- Considers overall cluster structure.

Group-Average Clustering

- Uses **average similarity of all document pairs between clusters.**
- **Balanced approach** between single-link and complete-link.

4. Illustrate the drawbacks of HAC clustering.

- **Computationally expensive** for large datasets
- Greedy algorithm: **wrong merges cannot be undone**
- Single-link suffers from **chaining**
- Complete-link is **sensitive to outliers**
- Requires **large memory for similarity matrix**

- **Slower** than K-means

5. What is the running time complexity of HAC? Discuss.

Naive HAC

$$O(N^3)$$

- Searches full similarity matrix in every iteration.

Improved HAC using Priority Queues

$$O(N^2 \log N)$$

- Faster similarity lookup and updates.

Single-Link HAC with NBM optimization

$$O(N^2)$$

- Possible because of best-merge persistence.

[will watch these before final !!] [**numericals**]

https://www.youtube.com/watch?v=pbTQQCA9Xs0
https://www.youtube.com/watch?v=Ufzq9oLhzX0
https://www.youtube.com/watch?v=tXYAdGn-SuM
https://www.youtube.com/watch?v=0A0wtto9wHU